

Assignment 2 — Feature Matching & Point Cloud Registration

This repository contains code for **Assignment 2** of the course **AER1515 - Perception for Robotics** at the University of Toronto. The assignment is divided into two main tasks:

1. **Feature Matching & Sparse Depth Estimation** (stereo vision pipeline with ORB, R2D2, and SuperGlue)
 2. **Point Cloud Registration** (ICP algorithm for 3D pose estimation)
-

Environment Setup

Prerequisites

- Linux (tested on Ubuntu 24.04)
- [Mamba](#) or [Conda](#)
- CUDA-capable GPU (optional, for GPU acceleration with R2D2/SuperGlue; CPU works but slower)

Installation

1. **Clone the repository** (if not already done):

```
git clone https://github.com/iolj-uoft/AER1515-Perception-for-Robotics.git
cd AER1515-Perception-for-Robotics/assignment2
```

2. **Create the environment from the provided YAML:**

```
mamba env create -f environment.yml
# or with conda:
# conda env create -f environment.yml
```

3. **Activate the environment:**

```
mamba activate AER1515
# or: conda activate AER1515
```

and install PyTorch 2.9.0 with CUDA 13.0:

```
pip3 install torch torchvision --index-url
https://download.pytorch.org/whl/cu130
```

4. Verify installation:

```
python -c "import torch; import cv2; import numpy; print('Environment OK')"
```

Part I — Feature Matching & Sparse Depth Estimation

Location: [Feature_Matching_Correspondence/](#)

This pipeline detects keypoints in stereo image pairs, matches them across left/right images, computes disparities, and estimates sparse depth maps.

Supported Methods

- **ORB** — Classical binary descriptors
- **R2D2** — Learned detector/descriptor
- **SuperGlue** — Learned matching with attention mechanism + Sinkhorn algorithm

Directory Structure

```
Feature_Matching_Correspondence/  
├─ starter_code_feature.py      # Main script for ORB / R2D2  
├─ superglue_feature.py        # Main script for SuperGlue pipeline  
├─ tools/  
│   ├── matcher.py             # Feature matching + RANSAC  
│   ├── R2D2.py                # R2D2 wrapper  
│   ├── superglue.py           # SuperGlue helper (batch processing)  
│   └─ calculate_depth_error.py # Depth evaluation metrics  
├─ r2d2/  
├─ SuperGluePretrainedNetwork/ # SuperGlue demo code  
├─ SuperGlueData/  
│   └─ images/                 # Copied/renamed images for SuperGlue  
├─ training/  
│   ├── left/                 # Left camera images  
│   ├── right/                # Right camera images  
│   ├── calib/                # Camera calibration files  
│   └─ gt_depth_map/          # Ground truth depth maps (for evaluation)  
├─ test/  
└─ P3_result.txt               # Output: predicted sparse depths
```

1. Running ORB or R2D2

Command:

```
cd Feature_Matching_Correspondence
python starter_code_feature.py --test 0 --plot 0 --use-RANSAC 1 --feature-detector R2D2
```

Arguments:

- `--test 0`: Use training set (with ground truth for evaluation); `--test 1`: use test set
- `--plot 0`: Disable visualizations (faster); `--plot 1`: save/show match plots
- `--use-RANSAC 1`: Enable RANSAC outlier rejection, otherwise will be using epipolar constraints
- `--feature-detector`: Choose `ORB` or `R2D2`

Examples:

```
# ORB
python starter_code_feature.py --test 0 --plot 0 --use-RANSAC 1 --feature-detector ORB

# R2D2
python starter_code_feature.py --test 0 --plot 1 --use-RANSAC 1 --feature-detector R2D2
```

Outputs:

- `P3_result.txt`: Predicted sparse depth values (format: `image_id u v depth`)
- (Optional) Match visualizations saved to current directory

2. Running SuperGlue

SuperGlue uses a two-step workflow:

1. **Batch matching**: Run the SuperGlue demo to generate `.npz` match files
2. **Depth computation**: Load `.npz` files and compute depths

Step 1: Run SuperGlue Demo (Batch Matching)

```
cd SuperGluePretrainedNetwork
python match_pairs.py \
  --input_pairs ../Feature_Matching_Correspondence/SuperGlueData/pairs.txt \
  --input_dir ../Feature_Matching_Correspondence/SuperGlueData/images \
  --output_dir ../Feature_Matching_Correspondence/SuperGlueData \
  --superglue outdoor \
  --resize -1 -1 \
  --viz
```

Key arguments:

- `--input_pairs`: Path to `pairs.txt` (lists image pairs to match)
- `--input_dir`: Folder containing images
- `--output_dir`: Where to save `.npz` match files
- `--superglue outdoor`: Use pretrained outdoor model (alternatives: `indoor`)
- `--resize -1 -1`: Disable resizing (preserves original pixel coordinates)
- `--viz`: Save match visualizations (optional)

Output: `.npz` files in `SuperGlueData/`, each containing:

- `keypoints0`, `keypoints1`: (N,2) arrays of (u,v) pixel coordinates
- `matches`: (N,) array of match indices
- `match_confidence`: (N,) array of confidence scores

Step 2: Compute Depths from SuperGlue Matches

```
cd Feature_Matching_Correspondence
python superglue_feature.py
```

What it does:

1. Loads `.npz` files and extracts matched keypoint pairs
2. Computes disparity $d = u_L - u_R$ and depth $Z = (f \times baseline) / d$
3. Filters by: disparity > 0.1 and $0 < Z \leq 80$ m
4. Writes valid predictions to `P3_result.txt` (requires ≥ 100 valid pairs per image)
5. Saves match visualizations (stacked vertically)

Outputs:

- `P3_result.txt`: Predicted sparse depths

Part II — Point Cloud Registration (ICP)

Location: `Point_Cloud_Registration/`

Implements the **Iterative Closest Point (ICP)** algorithm to align source and target 3D point clouds using nearest neighbor search and SVD-based pose estimation.

Directory Structure

```
Point_Cloud_Registration/
├─ starter_code_registration.py # Main ICP script
├─ make_plot.py                # Combine/stack plots (optional)
├─ training/                   # Training point clouds + ground truth
poses
├─ bunny_source.csv
├─ bunny_target.csv
├─ dragon_source.csv
└─ dragon_target.csv
```

```

├── test/                                # Test point cloud (no ground truth)
│   ├── armadillo_source.csv
│   └── armadillo_target.csv
└── figures/                             # Output: plots, poses, errors
    ├── <object>_before_registration.png
    ├── <object>_after_registration.png
    ├── <object>_icp_mean_distance.png
    ├── <object>_icp_translation_components.png
    ├── <object>_estimated_pose.txt
    ├── registration_errors.csv
    └── combined_*.png

```

Running ICP

Command:

```

cd Point_Cloud_Registration
python starter_code_registration.py

```

What it does:

1. Loads training/test point clouds (CSV format: x, y, z)
2. For each object:
 - Runs ICP for up to 30 iterations (or until convergence: mean distance < 1e-4)
 - Saves diagnostic plots (ICP loss, translation components per iteration)
 - Saves 3D scatter plots (before/after registration)
 - Computes rotation error (degrees) and translation error (L2 norm) vs. ground truth (training only)
 - Saves estimated 4×4 SE(3) pose matrix
3. Appends errors to `figures/registration_errors.csv`

Outputs (all saved at 300 dpi in `figures/`):

- **Plots:**
 - `{bunny,dragon,armadillo}_before_registration.png`
 - `{bunny,dragon,armadillo}_after_registration.png`
 - `{bunny,dragon,armadillo}_icp_mean_distance.png`
 - `{bunny,dragon,armadillo}_icp_translation_components.png`
- **Poses:** `{bunny,dragon,armadillo}_estimated_pose.txt` (4×4 matrix)
- **Errors:** `registration_errors.csv` (columns: object, rot_deg, trans_l2)

Example console output:

```

bunny -- Rotation error (deg): 0.0000, Translation error: 0.0001
dragon -- Rotation error (deg): 0.0000, Translation error: 0.0000
Estimated 6D pose for test sample 'armadillo':

```

```
[[ 0.777541 -0.066073  0.625351 -2.689722]
 [-0.384049  0.737557  0.555442 -1.858343]
 [-0.497931 -0.672045  0.548106 -5.961925]
 [ 0.         0.         0.         1.        ]]
```

Optional: Combine Plots

After running ICP, generate combined/stacked visualizations:

```
python make_plot.py
```

What it does:

- Vertically stacks per-object plots (e.g., all `*_icp_mean_distance.png`)
 - Creates horizontal before/after stitched images, then stacks them vertically
 - Saves combined images: `combined_icp_mean_distance.png`,
`combined_before_after_hstack.png`
-

Citations

- **R2D2**: Revaud et al., "R2D2: Repeatable and Reliable Detector and Descriptor", NeurIPS 2019
 - **SuperGlue**: Sarlin et al., "SuperGlue: Learning Feature Matching with Graph Neural Networks", CVPR 2020
 - **ORB**: Rublee et al., "ORB: an efficient alternative to SIFT or SURF", ICCV 2011
-

Contact

For questions or issues, please contact the course staff or open an issue in the repository.